

Practical Insights into Data Analysis and Machine Learning

PROGRAM - 9

Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

Objective

To apply unsupervised learning using the k-means clustering algorithm to group into clusters based on their features, and visualize how well the clustering separates target classes.

9. Introduction

Clustering is an essential unsupervised learning technique used to group similar data points based on their features without prior knowledge of labels. Among various clustering algorithms, k-means clustering is one of the most widely used due to its simplicity and efficiency. This program focuses on implementing the k-means clustering algorithm on the Wisconsin Breast Cancer Dataset, a widely used medical dataset containing features computed from digitized images of breast mass fine needle aspirates (FNA). The aim is to partition the dataset into clusters that ideally correspond to benign and malignant tumour cases based on feature similarities.

By applying k-means clustering, we seek to explore the underlying structure of the data and evaluate how well the unsupervised clusters correspond to the actual tumour classifications. Additionally, visualization techniques such as PCA (Principal Component Analysis) or t-SNE are used to project the high-dimensional data into two dimensions for better interpretability of the clustering results. This program demonstrates the power of clustering techniques in identifying patterns in medical data and highlights the potential of machine learning in aiding cancer diagnosis.

9.1 Clustering

- Clustering is a type of unsupervised machine learning technique that aims to group a set of data points into clusters, such that points in the same cluster are more similar to each other than to those in other clusters. It is widely used in exploratory data analysis to find natural groupings or patterns within a dataset without using predefined labels.
- In clustering, there is no target variable. The algorithm tries to infer the structure from the data. It is especially useful when the dataset lacks class labels or when discovering hidden patterns is the goal.

9.2 K-Means Clustering

K-Means Clustering is one of the most commonly used unsupervised learning algorithms. It is a centroid-based algorithm that aims to partition a dataset into K distinct, non-overlapping clusters. Each cluster is defined by its centroid (the mean of the data points in that cluster), and each data point belongs to the cluster with the nearest centroid.

K-Means Clustering Algorithm

- Choose the number of clusters (K): Decide how many clusters to identify in the data.
- Initialize centroids: Randomly select K data points from the dataset as initial cluster centroids.
- Assign data points to the nearest centroid:
 - For each data point, calculate the distance to each centroid.
 - Assign the point to the cluster with the closest centroid (commonly using Euclidean distance).
- Update centroids:
 - Recalculate the centroid of each cluster as the mean of all data points assigned to that cluster.
- Repeat steps 3 and 4 until:
 - The centroids no longer change (convergence), or
 - A maximum number of iterations is reached.

9.3 Program

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score

# Load the Wisconsin Breast Cancer dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y_true = data.target # true labels: 0 = malignant, 1 = benign

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply k-means clustering
kmeans = KMeans(n_clusters=2, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
```

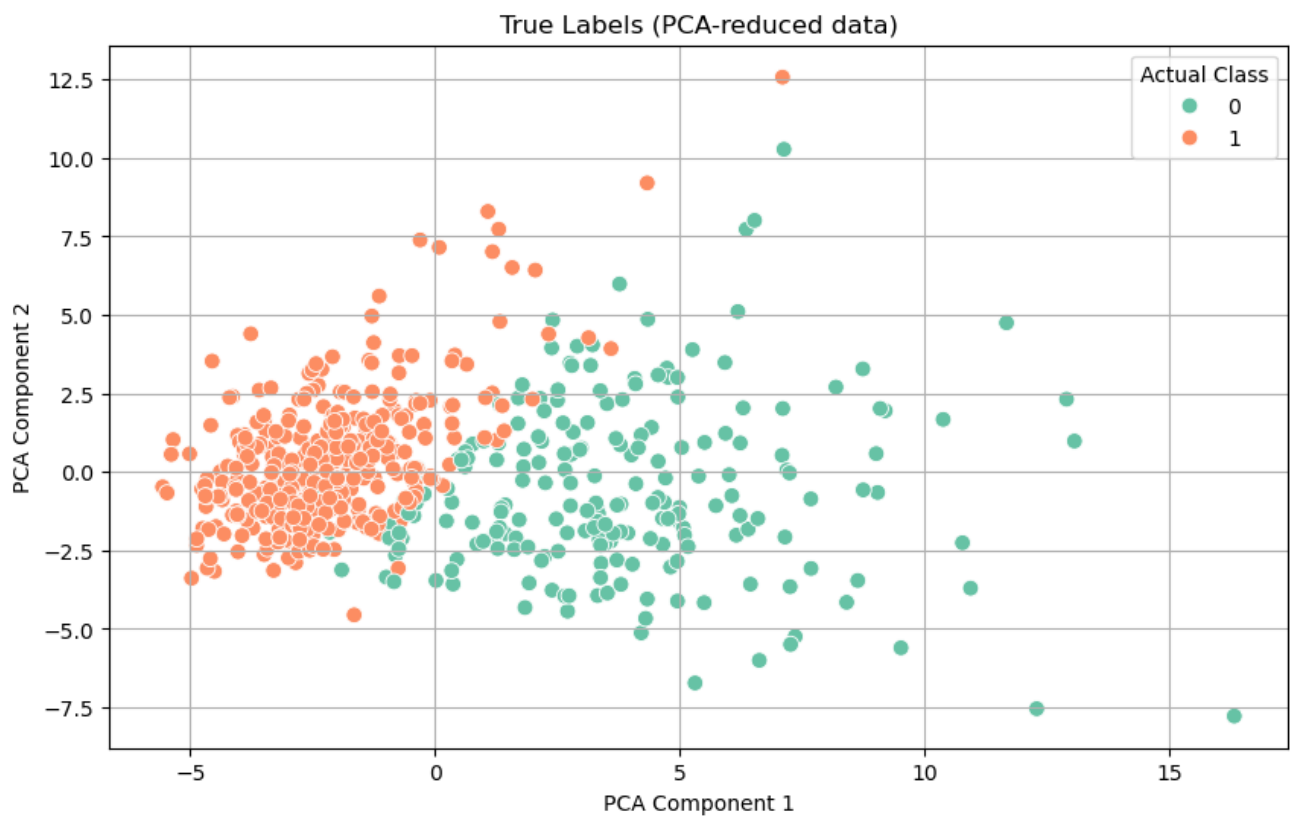
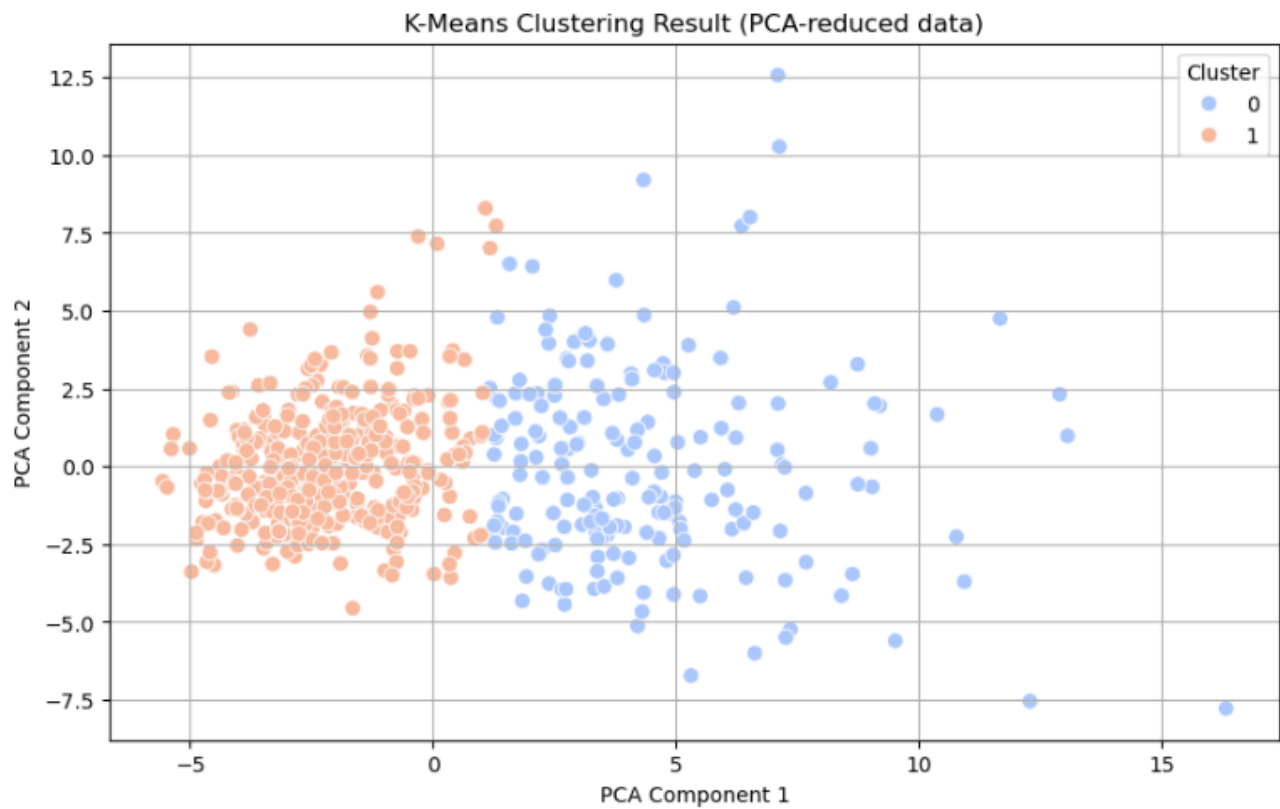
```
# Evaluate the model
silhouette_avg = silhouette_score(X_scaled, y_kmeans)
ari_score = adjusted_rand_score(y_true, y_kmeans)

print(f"Silhouette Score: {silhouette_avg:.3f}")
print(f"Adjusted Rand Index: {ari_score:.3f}")

# Visualize the clustering result using PCA (for 2D visualization)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(10,6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y_kmeans,
                palette="coolwarm", s=60)
plt.title('K-Means Clustering Result (PCA-reduced data)')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend(title="Cluster")
plt.grid(True)
plt.show()

# compare with the true labels
plt.figure(figsize=(10,6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y_true,
                palette="Set2", s=60)
plt.title('True Labels (PCA-reduced data)')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend(title="Actual Class")
plt.grid(True)
plt.show()
```



Viva Questions

- What is clustering in machine learning?
- How does k-means cluster work?
- Is k-means clustering supervised or unsupervised learning? Why?
- What kind of datasets is k-means suitable for?
- What is the role of the number of clusters (K) in k-means?
- How do you choose the optimal number of clusters in k-means?
- What distance metric is typically used in k-means?
- What happens if k is not chosen correctly?
- What is the difference between hard clustering and soft clustering?
- Explain the significance of centroids in the k-means algorithm.
- How do you evaluate the quality of clusters?
- How can you compare the clustering output with the actual labels?